TeraFlash pro HP Remote Access User Manual

1. Overview

This document describes how to remotely control the *TeraFlash pro HP* using a TCP socket interface. The remote access is divided into a control interface and a data stream interface. These interfaces allow remote access to the *TeraFlash pro control* software. Direct access to the hardware is not possible. The control interface is used to change system parameters, control the system, and read out current system parameters. The data stream interface is used to receive pulse measurement and spectral data from the system. To access each of these interfaces, two different ports are used (see **2. Network Configuration**).

For a quick setup, an example class is provided (*TFproRemoteAccess*) in Python, which combines the control socket and the stream socket into a single class providing a basic interface to send commands and receive measurement data (see **5. Quick Start**). For a more in-depth explanation of the interfaces, see (**3. Control Interface)** and (**4. Data Stream Interface**).

2. Network Configuration

Control Port (Send and receive SET/GET-Commands): **6009**
Stream Port (Stream Pulse/Spectrum data from device): **6008**
IP Address: Any IPv4 address of the computer on which the *TeraFlash pro control* software is running

3. Control Interface

The Control Port is used to control the system (e.g. start a measurement, turn on laser), set system parameters (e.g. measurement range, delay position, TIA setting) or read out the current system parameters.

3.1 How to use

1. Initialize socket connection to the Control Port (6009) to change or read out system parameters.
2. Send commands as UTF-8 encoded strings (See Command Reference 3.4 below)
3. **Important**: Always receive a response after sending a command (separated in 4-byte header containing the length of the response + payload).
4. Close connections after usage.

3.2 Response Format

For **GET**-Commands the response contains the requested parameter value. For **SET**-Commands the response contains the SET-Value which was applied (If a float value is rounded or was out of range, the read back returns the actual value which is set in the system). In case of an error one of the following error codes (**3.3 Error-Codes**) will be send back.

3.3 Error-Codes

Error-Code	Error-Type	Meaning
-1	**UNKNOWN_COMMAND_ERROR**	SET/GET-Command is not implemented
-2	**INVALID_ARGUMENTS_ERROR**	Provided argument for SET-command has a wrong format or is missing
-3	**CONNECTION_ERROR**	Connection to the system was lost
-4	**COMMAND_NOT_ALLOWED_ERROR**	SET-Command is currently not allowed due to safety reasons.

3.4. Command Reference

The commands that are sent via the control interface are divided in SET and GET Commands. The structure is the following:

GET/SET_{PARAMETER}:{VALUE}

Each command either starts with the **SET** or **GET** prefix followed by an underscore "_" and the parameter name. For **SET**-Commands the value that should be set is separated by a colon "**:**". The value can be either float, bool, string or int:

Example-Commands:

Description	Command	Value	Expected Response
Change range to 50ps	SET_RANGE:50	50	50
Change delay to 1304.2 ps	SET_DELAY:1304.2	1304.2	1304.2
Start a measurement	SET_MEASURE_ACTIVE:True	True	True
Turn on Laser	SET_LASER:True	True	True

3.5 Value Format Examples

For the types **string** and **bool** capitalization does not matter!
float: Use "." as decimal separator. Examples: 1.2345 or 5e6 (for 5000000)
bool: True/Enable: [true; TRUE; On; 1] or False/Disable: [false; FALSE; Off; 0]
string: See individual commands and use predefined names
int: See individual commands and use predefined values

3.6 List of SET/GET-Commands

GET_CONNECTED
Check connection state

SET_CONNECTED
Connect/disconnect system (bool)

GET_MEASURE_ACTIVE
Check measurement state

SET_MEASURE_ACTIVE
Start/stop measurement (bool)

SET_DELAY
Set delay position (float, ps)

GET_DELAY
Get delay position

SET_RANGE
Set shaker range
(int: [5,10,15,20,35,50,70,100,120,150,200], ps)

GET_RANGE
Get shaker range

SET_FAST_MODE
Enable fast shaker mode

SET_NORMAL_MODE
Enable normal shaker mode

GET_MODE
Get shaker mode

SET_TIA_MODE_1
Select external or internal TIA for channel 1 (string: [Internal, External])

SET_TIA_MODE_2
Select external or internal TIA for channel 2 (string: [Internal, External])

GET_TIA_MODE_1
Get TIA mode (internal or external) for channel 1

GET_TIA_MODE_2
Get TIA mode (internal or external) for channel 2

SET_TIA_GAIN
Set TIA gain for channel 1 and channel 2
(string: [low, mid, high])

GET_TIA_GAIN
Get TIA gain for channel 1 and channel 2

SET_EXT_TIA_GAIN_1
Set external TIA gain channel 1 (float)

GET_EXT_TIA_GAIN_1
Get external TIA gain channel 1

SET_EXT_TIA_GAIN_2
Set external TIA gain channel 2 (float)

GET_EXT_TIA_GAIN_2
Get external TIA gain channel 2

GET_AQU_MODE
Get acquisition mode

GET_BUSY
Check if system is busy

GET_ID
Get Software Version

SET_WAIT_FOR_AVERAGE_DONE
Enable to only receive data from data stream socket when current averaging window is complete.

GET_WAIT_FOR_AVERAGE_DONE
Get current selection
True: Only receive data when averaging window is full
False: Always receive data (if acquisition mode is either continuous or auto)

SET_AQU_MODE_CONT
Set acquisition mode to continuous

SET_AQU_MODE_WAIT
Set acquisition mode to wait

SET_AQU_MODE_AUTO
Set acquisition mode to automatic

SET_LASER
Enable/disable laser (bool)

GET_LASER
Get laser status

GET_LASER_ERROR
Get laser error

GET_LASER_READY
Check if laser ready

SET_EMITTER_1
Control emitter 1 (bool)

GET_EMITTER_1
Get emitter 1 status

SET_EMITTER_2
Control emitter 2 (bool)

GET_EMITTER_2
Get emitter 2 status

SET_AVERAGE
Set number of averages (int: [1;1000000])

GET_AVERAGE
Get current selected number of averages

GET_DATALENGTH
Get amount of data samples in one pulse trace

3.7 Example Usage (Python) (testControlSocket.py)

```python
from TFproRemoteAccess import ControlRemoteSocket

# Insert IP-Address of Computer where TeraFlash pro Control Software is running
IP_ADDRESS = "localhost"

ctrl = ControlRemoteSocket(ip=IP_ADDRESS)
ctrl.connect()
response = ctrl.send_command("GET_LASER")  # get state of the Laser
response = ctrl.send_command("GET_DELAY")  # read delay position
response = ctrl.send_command("SET_RANGE:20")  # change range to 20 ps
response = ctrl.send_command("SET_MEASURE_ACTIVE:True")  # turn on measurement
ctrl.close()
```

Hint: *See **5. Quick Start** for implementation of TFproRemoteAccess class providing higher level methods*

4. Data Stream Interface

The Data Stream Interface allows the user to receive live measurement data from the *TeraFlash pro control* software. While a measurement is active, the pulse and spectrum data displayed in the GUI are continuously published through the stream server and can be accessed via a TCP connection to the stream port.

When a client connects to the stream port, the server clears all internally buffered stream data and starts filling the buffers with newly acquired measurements. The client therefore receives only measurement data acquired after the connection was established. Measurement packets are transmitted continuously and contain the current pulse traces, spectra and averaging count.

4.1 How to use

1. Initialize socket connection to the Stream Port (**6008**)
2. Ensure that the acquisition mode is set to *Continuous*. If you want to receive data only when the averaging is completed, set the parameter SET_WAIT_FOR_AVERAGE_DONE to True.
3. Receive streamed data:
    1. Receive the first 4 bytes containing the metadata length (uint32, big-endian)
    2. Receive the metadata block (UTF-8 encoded JSON)
    3. Receive the raw data bytes by extracting the “total_bytes” from the metadata
    4. Reconstruct the individual arrays using the metadata information (offset, nbytes and dtype)

4.2 Stream Package Structure

Name	Type	Size	Description
Length-Header for meta data	Big endian uint32	4 Bytes	Contains the byte length **N** of the JSON metadata block
Metadata	UTF-8 JSON	**N** Bytes	Contains the metadata: “nAvgs”, “**total_bytes**”, ”array-metadata”: {“dtype”,”shape”,”offset”,”nbytes”} (one array-metadata per array
Data arrays (contains the measurement)	Raw Bytes	**total_bytes (from meta-data)**	concatenated C-contiguous array data

4.3 Unpacking the raw bytes into separate arrays:

The metadata describes where each array is located within the binary data block. For every array the corresponding byte range is extracted using the stored offset and size information:

start = info[“offset”]
stop = start + info[“nbytes”]
array = rawDataBlock[start:stop]

The extracted bytes are converted back into an array using the specified data type.

4.4 Example usage: Get streamed data (Python) (testStreamSocket.py)

```python
from TFproRemoteAccess import StreamRemoteSocket

# Insert IP-Address of Computer where TeraFlash pro Control Software is running
IP_ADDRESS = "localhost"

if __name__ == "__main__":
    streamSocket = StreamRemoteSocket(ip=IP_ADDRESS)
    streamSocket.connect()
    data = streamSocket.receive_stream()
    streamSocket.close()

    # get data from dictionary
    timeAxis = data["pulse"]["time/ps"]
    pulseTrace1 = data["pulse"]["channel1/nA"]
    freq1 = data["spectrum"]["frequency1/THz"]
    spec1 = data["spectrum"]["channel1/dB"]
```

Step 4.1 to 4.3 from receiving the stream package to conversion into arrays is already implemented inside the *receive_stream()* method.

Hint: *See chapter **5. Quick Start** for implementation of TFproRemoteAccess class providing higher level methods regarding different ways to acquire data via remote access.*

4.5 Received data structure of receive_stream() method

The received data (dictionary) has the following structure:

{“pulse” : { “time/ps”: [...],“channel1/nA”: [...] , “channel2/nA: [...]}, “spectrum”: {“frequency1/THz”:[...],”frequency2/THz:[...], “channel1/dB”: [...] , “channel2/dB”:[...] }}

Data-Set	Parent-Key	Trace-Key	Dictionary Access
time axis of pulse trace for channel 1 and channel 2	pulse	time/ps	data[“pulse”][“time/ps”]
pulse trace channel 1	pulse	channel1/nA	data[“pulse”][“channel1/ps”]
pulse trace channel 2	pulse	channel2/nA	data[“pulse”][“channel2/ps”]
frequency axis channel 1	spectrum	frequency1/THz	data[“spectrum”][“ frequency1/THz ”]
frequency axis channel 2	spectrum	frequency2/THz	data[“spectrum”][“ frequency2/THz ”]
spectrum channel 1	spectrum	channel1/dB	data[“spectrum”][“ channel1/dB ”]
spectrum channel 2	spectrum	channel2/dB	data[“spectrum”][“ channel2/dB ”]

5. Quick Start

For an easy start, use the provided Python class ***TFproRemoteAccess.py***. This class combines the Control Interface with the Data Stream Interface. The main communication protocol is implemented there. The class can be used as-is or can serve as a guideline for implementing more complex tasks.

5.1 List of the implemented methods inside TFproRemoteAccess class

Method	Arguments	Description
connect_device	-	Connect the GUI with the TeraFlash pro (Same behavior then clicking "Connect" button inside GUI)
get_id	-	Read out version of TeraFlash pro Control Software
send_command	command (string): raw command	Send a raw command string to the control server and return the response. Use this for commands not covered by the high-level methods. See (3.6 List of SET/GET-Commands)
start_measurement	-	Activate the measurement and set the acquisition mode to continuous.
get_averaged_measurement	nAvgs (integer): number of averages to accumulate restart_averaging (bool): restart averaging before acquisition starts	Acquire a single fully averaged measurement. Waits until *nAvgs* averages have accumulated before the averaged data is returned.
get_averaging_sequence	nAvgs (integer): number of averages to accumulate	Acquire all intermediate traces as the averaging window builds up. Triggers a fresh averaging run and returns one packet per accumulation step, i.e. the packet with nAvgs=1, then nAvgs=2, up to the final nAvgs=N. The total number of packets returned equals nAvgs.
get_continuous_traces	n_traces (integer): number of consecutive traces to receive wait_for_average_done: the data is only send when a full averaging window is completed.	Acquire a sequence of consecutively measured traces in continuous mode.
restart_averaging	-	Reset the averaging window and immediately resume continuous acquisition. Equivalent to triggering AUTO mode and then switching back to continuous mode.
close	-	Close both the stream and control socket connections. Should be called when the session is complete to release OS socket resources.

5.2 Example code for using the TFproRemoteAccess class:

Initiate an object by providing the IP address of the computer where the *TeraFlash pro control* software is running. See the ***testTFproRemoteAccess.py*** script for a small example application.

```python
# import matplotlib.pyplot as plt  # uncomment to plot results
from TFproRemoteAccess import (
    TFproRemoteAccess,
    PULSE_CH1,
    PULSE_CH2,
    PULSE_X_AXIS,
    SPEC_CH1,
    SPEC_CH2,
    SPEC_X_AXIS1,
    SPEC_X_AXIS2,
    PULSE_KEY,
    SPEC_KEY,
    N_AVGS_KEY,
)

# Insert IP-Address of Computer where TeraFlash pro Control Software is running
IP_ADDRESS = "localhost"

if __name__ == "__main__":
    # Create Remote-Access Socket and connect to Control-Socket
    tf = TFproRemoteAccess(ip=IP_ADDRESS, verbose=True)

    # connect to the TFpro
    tf.connect_device()

    # start a measurement
    tf.start_measurement()

    # change shaker range to 20ps
    response = tf.send_command("SET_RANGE:20")

    # take single measurement with 100 averages
    data = tf.get_averaged_measurement(nAvgs=100)

    # Access recorded data from get_averaged_measurement
    pulseXAxis = data[PULSE_KEY][PULSE_X_AXIS]
    pulseCh1 = data[PULSE_KEY][PULSE_CH1]
    pulseCh2 = data[PULSE_KEY][PULSE_CH2]
    specXAxis1 = data[SPEC_KEY][SPEC_X_AXIS1]
    specXAxis2 = data[SPEC_KEY][SPEC_X_AXIS2]
    specCh1 = data[SPEC_KEY][SPEC_CH1]
    specCh2 = data[SPEC_KEY][SPEC_CH2]
    nAvgs = data[N_AVGS_KEY]

    # fig, ax = plt.subplots(2)
    # ax[0].plot(pulseXAxis, pulseCh2, label="ch2")
    # ax[0].plot(pulseXAxis, pulseCh1, label="ch1")
    # ax[0].legend()
    # ax[1].plot(specXAxis2, specCh2, label="ch2")
    # ax[1].plot(specXAxis1, specCh1, label="ch1")
    # ax[1].legend()

    # receive every package while averaging up to nAvgs
    dataPackets = tf.get_averaging_sequence(nAvgs=10)

    # Close Connection
    tf.close()

    # fig, ax = plt.subplots(2)
    ## Access recorded data from get_averaging_sequence measurement
    # for data in dataPackets:
    #     pulseXAxis = data[PULSE_KEY][PULSE_X_AXIS]
    #     pulseCh1 = data[PULSE_KEY][PULSE_CH1]
    #     specXAxis1 = data[SPEC_KEY][SPEC_X_AXIS1]
    #     specCh1 = data[SPEC_KEY][SPEC_CH1]
    #     nAvgs = data[N_AVGS_KEY]
    #     ax[0].plot(pulseXAxis, pulseCh1, label=f"{nAvgs}")
    #     ax[0].legend()
    #     ax[1].plot(specXAxis1, specCh1, label=f"{nAvgs}")
    #     ax[1].legend()
    # plt.show()
```